SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	G. Mohammed Shariff	**Reg. No.:**	192425378
Date:	21/08/2026	**Duration:**	60 minutes

Code of Conduct
I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: _____

Annexure A – CI/CD Pipeline Design Exercise

TITLE:Al-Based Learning Management System with Personalized Learning Recommendations

DOMAIN:EDUCATION

1. Project Overview and CI/CD Requirements Analysis

1.1 System Overview

The AI-Based Learning Management System (LMS) is an education platform that allows students to enrol in courses, consume learning content, attempt assessments, and receive personalized course/ content recommendations generated by a machine learning (ML) recommendation engine. The system is composed of the following major components:

- **Frontend Web/Mobile App:** React-based student and instructor portal for course access, dashboards, and recommendations display.
- **Backend API Service:** REST/GraphQL service handling authentication, course management, enrolment, and assessment logic.
- **Recommendation Engine (ML Service):** A Python-based microservice that trains and serves collaborative-filtering/content-based models to suggest courses, learning paths, and resources personalised to each learner.
- **Database & Data Warehouse:** Relational database for transactional data (users, courses, grades) and a data store for learner interaction logs used to retrain ML models.
- **Notification Service:** Sends email/push alerts for deadlines, new recommendations, and grades.

1.2 CI/CD Needs Identified

Given the above architecture, the following CI/CD requirements were identified:

- **Independent, frequent releases:** The frontend, backend API, and ML recommendation service evolve at different speeds and must be built, tested, and deployed independently as containerised microservices.
- **Separate ML pipeline:** The recommendation model requires its own validation pipeline (accuracy, drift, fairness checks) in addition to standard code testing, since a degraded model directly harms the learning experience.
- **High reliability during active terms:** The LMS is used continuously during the academic term (assessments, deadlines); the pipeline must support zero/low-downtime deployment and fast rollback.
- **Data privacy and security:** The system stores student personal and academic data, so the pipeline must enforce security scanning, secrets management, and compliance-aware handling of data (FERPA/GDPR-like requirements).
- **Scalability validation:** Automated performance testing is required to ensure the platform handles concurrent load during peak periods such as exam weeks.
- **Fast, safe experimentation:** The pipeline must support canary/A-B rollout of new recommendation models so that new ML versions can be validated on a small user segment before full rollout.

2. CI/CD Pipeline Architecture and Workflow Stages

The pipeline is organised into six logical stages that automate the journey of a code or model change from developer commit to production release:

Stage	Purpose	Trigger
1. Source Code Management	Version control of application code, ML training code, and infrastructure-as-code; enforces branching policy and peer review.	Developer push / Pull Request
2. Build	Compiles/packages each microservice (frontend, backend, ML service) and builds Docker images.	On every commit / PR to develop
3. Automated Testing	Runs unit, integration, ML-model validation, and security tests.	Automatically after successful build
4. Code Quality & Security Analysis	Static code analysis, dependency and container vulnerability scanning, quality gate enforcement.	Automatically after tests pass
5. Packaging & Artifact Storage	Tags and pushes versioned Docker images/model artifacts to a registry.	After quality gate passes
6. Continuous Deployment	Deploys the packaged artifact progressively to Dev, Staging, and Production environments with approval gates.	Merge to release/main branch

Branching strategy: the project follows a GitFlow-style model — feature branches → develop (integration) → release branch (staging validation) → main (production). Each merge type triggers a different depth of the pipeline, keeping feedback fast for feature branches and thorough for release/main branches.

3. Tools and Technologies per Stage (with Justification)

Pipeline Stage	Tool(s)	Justification
Source Code Management	Git + GitHub	Industry-standard distributed VCS; GitHub provides pull requests, branch protection rules, and native webhook integration to trigger CI.
CI Orchestration	GitHub Actions (Jenkins as alternative)	Native integration with GitHub repos, YAML-based pipeline-as-code, free for education use, and large marketplace of pre-built actions.
Build	Docker (multi-stage builds), Maven/npm/pip per service	Containerising each microservice guarantees environment parity between dev, staging, and production and simplifies deployment to Kubernetes.
Unit & Integration Testing	JUnit/PyTest (backend & ML), Jest + React Testing Library (frontend), Postman/Newman (API contracts)	Framework matches each service's language; Newman allows API contract tests to run headlessly inside the pipeline.
ML Model Validation	MLflow (experiment tracking & model registry), Great Expectations (data validation)	MLflow versions models and metrics so a new model is only promoted if it beats the accuracy/precision threshold of the current production model; Great Expectations validates the training data schema before retraining.
Code Quality Analysis	SonarQube	Detects code smells, duplication, and enforces a minimum test-coverage quality gate before merge.

Security Scanning	Snyk / OWASP Dependency-Check (dependencies), Trivy (container images), OWASP ZAP (DAST)	Layered scanning covers vulnerable libraries, insecure container base images, and runtime vulnerabilities — important given student PII stored by the LMS.
Artifact Repository	Docker Registry (Amazon ECR) + JFrog Artifactory (build artifacts/model files)	Central, versioned storage for immutable, traceable build artifacts that are promoted between environments rather than rebuilt.
Infrastructure as Code	Terraform + Kubernetes manifests/Helm charts	Enables reproducible, version-controlled environment provisioning across Dev/ Staging/Production.
Deployment / Orchestration	Kubernetes (EKS/GKE) + Argo CD (GitOps)	Kubernetes gives auto-scaling and self-healing for peak student load; Argo CD gives declarative, auditable, one-click rollback deployments.
Secrets Management	HashiCorp Vault / Kubernetes Secrets	Keeps database credentials and API keys out of source code and CI logs.
Monitoring & Logging	Prometheus + Grafana, ELK/ EFK stack, Sentry	Real-time metrics/dashboards and centralised logs are essential to detect a degraded recommendation model or service outage quickly.
Notifications	Slack/MS Teams webhooks, Email/PagerDuty	Immediate visibility of pipeline failures or production incidents to the responsible team.

4. Automated Testing Strategy

A layered testing strategy (test pyramid) is integrated into the pipeline so that faster, cheaper tests run first and give quick feedback, while slower, broader tests run later before production release:

- **Unit Testing:** Runs on every commit for each microservice (JUnit/PyTest/Jest). Verifies individual functions such as grade calculation, enrolment logic, and the recommendation-scoring function in isolation. Target coverage ≥ 80%, enforced by the SonarQube quality gate.

- **Integration Testing:** Validates interaction between backend API, database, and the recommendation service (e.g., does the API correctly fetch and display a model's recommendation output). Run using Postman/Newman/REST-assured against a containerised test environment.
- **ML-Specific Testing:** Includes data-schema validation (Great Expectations), model accuracy/ precision/recall regression checks against a held-out validation set, and data-drift detection so a retrained model is only promoted if it meets or exceeds the current production model's benchmark.
- **Regression / UI Testing:** Automated Selenium/Cypress test suite covering critical student journeys — login, course enrolment, assessment submission, and viewing recommendations — to catch breakages introduced by unrelated changes.
- **Performance & Load Testing:** k6/JMeter scripts simulate concurrent student traffic (e.g., simultaneous quiz submissions during exam week) run against the staging environment before production promotion.
- **Security Testing:** SAST via SonarQube, dependency scanning via Snyk, and DAST via OWASP ZAP against the staging deployment, checking for vulnerabilities such as SQL injection, XSS, and insecure authentication.

5. Deployment Strategy

Environment	Purpose	Deployment Trigger	Strategy
Development (Dev)	Rapid, continuous verification of new features by developers in an isolated namespace.	Automatic, on every merge to the develop branch.	Rolling deployment; ephemeral Kubernetes namespace, minimal manual gating.
Testing / Staging	A production-like environment for integration, performance, security, and UAT before release.	Automatic, on merge to a release branch.	Full replica of production infrastructure using anonymised/synthetic student data.
Production	Live environment used by students and instructors.	Manual approval gate after staging sign-off; merge to main branch.	Blue-green deployment for the core LMS application to guarantee zero downtime, combined with canary releases (5–10% of traffic first) for new recommendation-engine model versions, progressively increased while key metrics (click-through rate, latency, error rate) are monitored.

This staged promotion (Dev → Staging → Production) ensures that a change is validated in progressively more realistic conditions before reaching real students, while the blue-green/canary approach for production specifically protects the platform from a poorly performing recommendation model or a faulty release during active academic terms.

6. Security, Quality Checks, Notifications, and Rollback Mechanisms

6.1 Security Practices

- **Secrets management:** Database credentials, API keys, and model registry tokens are stored in HashiCorp Vault/Kubernetes Secrets, never in source code or pipeline logs.
- **Image & dependency scanning:** Every container image is scanned with Trivy and every dependency with Snyk/OWASP Dependency-Check before it can be promoted.
- **Data protection:** Student personal and academic data is encrypted in transit (TLS) and at rest, with access controlled through role-based access control (RBAC) on Kubernetes namespaces, in line with FERPA/GDPR-style data-protection expectations.
- **Signed artifacts:** Container images are signed and verified before deployment to prevent tampering.

6.2 Quality Gates

- **Branch protection:** Pull requests to develop/main require passing CI checks and at least one peer review before merge.
- **SonarQube quality gate:** Blocks promotion if test coverage falls below 80% or if new critical/blocker code smells or vulnerabilities are introduced.
- **Model promotion gate:** A retrained recommendation model is only promoted to staging/production if MLflow-tracked evaluation metrics meet or exceed the current production model's benchmark.

6.3 Notifications

The pipeline sends automated notifications via Slack/MS Teams webhooks at build start, on test/quality-gate failure, and on successful deployment; production deployments additionally notify the engineering lead by email, and PagerDuty alerts are raised for production incidents detected by monitoring.

6.4 Rollback Mechanisms

- **Application rollback:** Kubernetes/Argo Rollouts automatically reverts to the previous stable version (kubectl rollout undo) if post-deployment health checks or error-rate/latency metrics degrade beyond a set threshold during a canary or blue-green release.
- **Database rollback:** Schema changes are managed with versioned, reversible migrations (Flyway/Liquibase) so a release can be rolled back without data loss.
- **Model rollback:** The recommendation engine reads its active model version from the MLflow model registry; a bad model can be reverted to the previous registered version instantly.
- **Feature flags:** Non-critical features, including new recommendation algorithms, are wrapped in feature flags (e.g., LaunchDarkly) so they can be disabled instantly in production without a full redeployment.

7. Workflow Explanation

Workflow Explanation: From Commit to Deployment

- **Step 1 — Commit & Trigger:** A developer pushes code (application code, or updated ML training script) to a feature branch and opens a pull request into develop. GitHub's webhook automatically triggers the CI pipeline in GitHub Actions.
- **Step 2 — Build:** The pipeline builds Docker images for the affected microservice(s) — frontend, backend API, or ML recommendation service — using multi-stage Docker builds.
- **Step 3 — Automated Testing:** Unit tests run first for fast feedback, followed by integration tests. If the change touches the recommendation engine, the ML validation stage retrains/evaluates the model against the benchmark dataset and checks accuracy and data-drift thresholds.
- **Step 4 — Code Quality & Security Analysis:** SonarQube performs static analysis and enforces the quality gate; Snyk/Trivy scan dependencies and container images for vulnerabilities. A failure at this stage stops the pipeline and notifies the developer via Slack.
- **Step 5 — Packaging:** On success, the versioned Docker image (and, if applicable, the new model artifact) is pushed to the container/model registry, ready for deployment.
- **Step 6 — Deploy to Dev:** Argo CD automatically deploys the new artifact to the Dev environment for immediate developer verification.
- **Step 7 — Promote to Staging:** Once the release branch is merged, the same artifact (not rebuilt) is promoted to Staging, where integration, performance/load, security (DAST), and UAT testing are performed against production-like infrastructure and data.
- **Step 8 — Manual Approval Gate:** A release manager reviews staging test results and metrics dashboards and manually approves promotion to production — a deliberate control point given the platform's active use by students.
- **Step 9 — Deploy to Production:** The core LMS application is released using a blue-green switch for zero downtime; a new recommendation-model version is rolled out as a canary to a small percentage of users first.
- **Step 10 — Monitor & Rollback:** Prometheus/Grafana and Sentry continuously monitor error rates, latency, and recommendation click-through rate. If metrics degrade beyond the configured threshold, Argo Rollouts automatically rolls back to the last stable version and the on-call engineer is alerted via PagerDuty/Slack.

This end-to-end pipeline ensures that every change to the AI-Based Learning Management System — whether an application feature or a new personalized-recommendation model — is automatically built, rigorously tested, security-checked, and released to students in a controlled, observable, and reversible manner.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/ rollback aspects	Basic deployment strategy with limited security/ rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation(10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/ documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	**Mark**
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25